Python E-Course

Module 13 — Feature Detection & Matching

Detailed Notes

Module Overview

Level: Features

Duration: ~1.5 weeks

Prerequisites: Modules 1-12

What You'll Learn

- Detect templates with cv2.matchTemplate.
- Detect corners with Harris and goodFeaturesToTrack.
- Use ORB to compute keypoints + descriptors.
- Match descriptors between two images with BFMatcher.
- Detect lines and circles with the Hough transform.

Lessons

#	Lesson	Duration
1	Template Matching	25 min
2	Harris & Corner Detection	30 min
3	ORB Keypoints & Descriptors	35 min
4	Feature Matching	35 min
5	Hough Transform — Lines & Circles	35 min

Assessment

- Exercises - Quiz - Assignment - Mini Project

What's Next

Module 14: Frequency Domain (DFT)

Lesson 1: Template Matching

Module: 13 — Feature Detection & Matching

Duration: 25 minutes

Difficulty: Beginner-Intermediate

Learning Objectives

- Use cv2.matchTemplate to find a small template inside a larger image.
- Pick the right matching method.
- Recognise template matching's limitations (rotation, scale).

Prerequisites

- Modules 1-12

1. Why This Matters

When the thing you're looking for has a fixed appearance — a logo, a button, an icon — template matching is the fastest, simplest detector. It's the first thing to try before reaching for ORB or deep learning.

2. Concept

2.1 The idea

Slide a small template image over a larger image, compute a similarity score at every position. The brightest (or darkest, depending on method) location is the match.

2.2 The function

```
result = cv2.matchTemplate(image, template, method)
```

result has shape (H - h + 1, W - w + 1) — a scalar score per valid position.

2.3 Match methods

Method	Best =	Notes
cv2.TM_SQDIFF	minimum	sum of squared differences
cv2.TM_SQDIFF_NORMED	minimum	normalised SQDIFF
cv2.TM_CCORR	maximum	cross-correlation
cv2.TM_CCORR_NORMED	maximum	normalised CCORR
cv2.TM_CCOEFF	maximum	mean-centred correlation
cv2.TM_CCOEFF_NORMED	maximum	best default — invariant to brightness/contrast

Use TM_CCOEFF_NORMED unless you have a reason not to.

2.4 Finding the match

```
min_val, max_val, min_loc, max_loc = cv2.minMaxLoc(result)
# For CCOEFF_NORMED: top-left of best match is max_loc
```

```
top_left = max_loc
bottom_right = (top_left[0] + tw, top_left[1] + th)
```

2.5 Multi-match

For more than one match, threshold the result map:

```
threshold = 0.85
ys, xs = np.where(result >= threshold)
for x, y in zip(xs, ys):
    cv2.rectangle(image, (x, y), (x + tw, y + th), (0, 255, 0), 2)
```

2.6 Limitations

Template matching fails when the target is:

- Rotated (works only at the rotation in the template).
- Scaled (size must match).
- Occluded (partial occlusion drops the score).
- Photometrically different (use _NORMED variants to help).

For invariance, use ORB / SIFT (next lessons).

3. Code Examples

Example 1: Find an eye in the astronaut image

```
import cv2
from skimage.data import astronaut

img = cv2.cvtColor(astronaut(), cv2.COLOR_RGB2BGR)
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

# Template: crop a 50x50 region around the right eye
template = gray[220:270, 260:310].copy()
print("template shape:", template.shape)

result = cv2.matchTemplate(gray, template, cv2.TM_CCOEFF_NORMED)
_, max_val, _, max_loc = cv2.minMaxLoc(result)
print("best match score:", max_val, "at", max_loc)

h, w = template.shape
cv2.rectangle(img, max_loc, (max_loc[0] + w, max_loc[1] + h), (0, 255, 0), 2)
cv2.imwrite("astro_template_match.png", img)
```

Expected Output: Green box drawn around the original eye location (the template was cut from there, so best match is exactly the original spot with score ~1.0).

Example 2: Multi-match — find all instances

```
import cv2, numpy as np

# Build a synthetic scene with 3 copies of a small "X"
scene = np.full((300, 600), 200, dtype=np.uint8)
```

```
template = np.full((30, 30), 50, dtype=np.uint8)
cv2.line(template, (3, 3), (27, 27), 255, 3)
cv2.line(template, (27, 3), (3, 27), 255, 3)

for x, y in [(50, 50), (300, 100), (450, 200)]:
    scene[y:y+30, x:x+30] = template

result = cv2.matchTemplate(scene, template, cv2.TM_CCOEFF_NORMED)
threshold = 0.95
ys, xs = np.where(result >= threshold)
print("matches found:", len(xs))
out = cv2.cvtColor(scene, cv2.COLOR_GRAY2BGR)
for x, y in zip(xs, ys):
    cv2.rectangle(out, (x, y), (x + 30, y + 30), (0, 255, 0), 2)
cv2.imwrite("multi_match.png", out)
```

Expected Output: 3 boxes drawn around the 3 placed Xs. (May be slightly more than 3 if neighbouring pixels also exceed threshold; can be deduplicated with NMS.)

Example 3: Compare match methods

```
import cv2
from skimage.data import astronaut

gray = cv2.cvtColor(cv2.cvtColor(astronaut(), cv2.COLOR_RGB2BGR),
cv2.COLOR_BGR2GRAY)
tpl  = gray[220:270, 260:310]

for name, method, find in [
    ("SQDIFF",         cv2.TM_SQDIFF,         "min"),
    ("CCORR_NORMED",   cv2.TM_CCORR_NORMED,   "max"),
    ("CCOEFF_NORMED",  cv2.TM_CCOEFF_NORMED,  "max"),
]:
    res = cv2.matchTemplate(gray, tpl, method)
    mn, mx, mn_loc, mx_loc = cv2.minMaxLoc(res)
    print(f"{name:14s} best={'min' if find=='min' else 'max'} loc {mn_loc if
find=='min' else mx_loc}")
```

Expected Output: All three methods locate the eye in approximately the same place (around (260, 220)).

4. Parameter Sensitivity

Param	Effect
Match method	NORMED variants handle brightness shifts
Threshold (for multi-match)	lower = more matches (incl false positives)
Template size	bigger = stricter match but slower

5. Common Mistakes

Mistake	What Happens	Fix
Search rotated template	Misses	Try ORB / SIFT
Template size doesn't match	Misses	Multi-scale: resize template

scene scale		at various scales
Use raw CCORR on varying brightness	False positives	Use _NORMED variants
Find min instead of max for CCOEFF	Wrong location	Check direction per method

6. Practice Tasks

- Crop a region from any image and use it as a template; find its location.
- Multi-match for repeated icons in a synthetic scene.
- Compare 3 match methods on the same input.

7. Key Takeaways

- Template matching = slide template, score, find best.
- cv2.TM_CCOEFF_NORMED is the safe default.
- Threshold the score map for multi-match.
- No rotation/scale invariance — use ORB for that.

8. What's Next

Harris corner detection — find "interesting" points without a template.

Lesson 2: Harris & Corner Detection

Module: 13 — Feature Detection & Matching

Duration: 30 minutes

Difficulty: Intermediate

Learning Objectives

- Apply cv2.cornerHarris and cv2.goodFeaturesToTrack.
- Understand why corners are good features (gradient in 2 directions).
- Tune parameters for typical scenes.

Prerequisites

- Module 13 Lesson 1

1. Why This Matters

A corner is a pixel where intensity changes sharply in two directions — distinctive enough to track across frames, recognise across images, or use as a keypoint for higher-level features (ORB, SIFT). Harris was the first major corner detector; Shi-Tomasi (cv2.goodFeaturesToTrack) is its everyday successor.

2. Concept

2.1 What makes a corner

- Flat region: gradient ≈ 0 in any direction. Not a corner.
- Edge: gradient large in one direction (perpendicular to edge). Not a corner.
- Corner: gradient large in two perpendicular directions.

2.2 Harris response

For each pixel, Harris computes a response based on the gradient covariance matrix:

```
R = det(M) − k · trace(M)²
```

Where M is built from local Sobel gradients. Pixels with R above a threshold are corners.

2.3 OpenCV API — Harris

```
gray = np.float32(gray)
dst = cv2.cornerHarris(gray, blockSize=2, ksize=3, k=0.04)
dst = cv2.dilate(dst, None)
mask = dst > 0.01 * dst.max()
img[mask] = (0, 0, 255)
```

Param	Meaning
blockSize	Neighbourhood size for M (2 default)
ksize	Sobel aperture (3)
k	Harris parameter (0.04-0.06)

2.4 Shi-Tomasi (goodFeaturesToTrack)

Easier-to-use, more reliable variant. Returns the top-N strongest corners directly:

```
corners = cv2.goodFeaturesToTrack(gray, maxCorners=100, qualityLevel=0.01,
minDistance=10)
```

Param	Meaning
maxCorners	maximum number to return
qualityLevel	fraction of best corner's quality required (0.01 = corners ≥ 1% of best)
minDistance	min pixels between corners (anti-clustering)

2.5 Sub-pixel refinement

For precise corner positions:

```
corners = cv2.goodFeaturesToTrack(gray, 100, 0.01, 10)
criteria = (cv2.TERM_CRITERIA_EPS + cv2.TERM_CRITERIA_MAX_ITER, 30, 0.001)
refined = cv2.cornerSubPix(gray, corners, (5, 5), (-1, -1), criteria)
```

2.6 Pre-blur

Like every gradient-based op, light Gaussian blur (σ=1) before corner detection makes results more robust.

3. Code Examples

Example 1: Harris on a checkerboard

```
import cv2, numpy as np
from skimage.data import checkerboard

img = checkerboard().astype(np.uint8) * 255 if checkerboard().max() <= 1 else
checkerboard().astype(np.uint8)
gray = np.float32(img)
dst = cv2.cornerHarris(gray, blockSize=2, ksize=3, k=0.04)
dst = cv2.dilate(dst, None)

vis = cv2.cvtColor(img, cv2.COLOR_GRAY2BGR)
vis[dst > 0.01 * dst.max()] = (0, 0, 255)
cv2.imwrite("checker_harris.png", vis)
print("max response:", dst.max())
```

Expected Output: Checkerboard with red dots at the corners of the squares.

Example 2: goodFeaturesToTrack

```
import cv2, numpy as np
from skimage.data import astronaut

gray = cv2.cvtColor(cv2.cvtColor(astronaut(), cv2.COLOR_RGB2BGR),
cv2.COLOR_BGR2GRAY)
corners = cv2.goodFeaturesToTrack(gray, maxCorners=100, qualityLevel=0.01,
minDistance=10)
```

```
vis = cv2.cvtColor(gray, cv2.COLOR_GRAY2BGR)
for c in corners.astype(int):
    x, y = c.ravel()
    cv2.circle(vis, (x, y), 3, (0, 255, 0), -1)
cv2.imwrite("astro_corners.png", vis)
print("corners found:", len(corners))
```

Expected Output: Astronaut image with 100 green dots at strong corners — eyes, helmet edges, mission patch text.

Example 3: Tune qualityLevel

```
import cv2, numpy as np
from skimage.data import camera

gray = camera()
for q in [0.005, 0.01, 0.05, 0.2]:
    c = cv2.goodFeaturesToTrack(gray, 200, q, 10)
    print(f"q={q}: {len(c) if c is not None else 0} corners")
```

Expected Output: Higher q → fewer corners (only the strongest). q=0.2 typically returns only a handful.

Example 4: Sub-pixel refinement

```
import cv2, numpy as np
from skimage.data import checkerboard

img = (checkerboard().astype(np.uint8) * 255) if checkerboard().max() <= 1 else
checkerboard()
gray = np.float32(img)

corners = cv2.goodFeaturesToTrack(gray, 81, 0.01, 10)
criteria = (cv2.TERM_CRITERIA_EPS + cv2.TERM_CRITERIA_MAX_ITER, 30, 0.001)
refined = cv2.cornerSubPix(gray, corners, (5, 5), (-1, -1), criteria)
print("raw corner 0:", corners[0].ravel())
print("refined    0:", refined[0].ravel())
```

Expected Output: Refined coordinates are slightly different — typically sub-pixel adjustments like (20.000, 20.000) → (19.998, 19.998).

4. Parameter Sensitivity

Param	Effect
qualityLevel	higher = stricter / fewer corners
minDistance	spacing between detections
k (Harris)	rarely changed; 0.04-0.06

5. Common Mistakes

Mistake	What Happens	Fix
Pass uint8 to cornerHarris	Should be float	np.float32(gray)

qualityLevel too low	Lots of false corners	Try 0.01-0.05
Forget to dilate Harris dst	Tiny single-pixel hotspots	cv2.dilate(dst, None) for visibility
Use as descriptor	Just coordinates, no descriptor	Use ORB/SIFT for descriptors

6. Practice Tasks

- Harris on a checkerboard; show red dots at corners.
- goodFeaturesToTrack (top 100) on astronaut().
- Compare qualityLevel ∈ {0.005, 0.05, 0.2}.

7. Key Takeaways

- Corner = strong gradient in 2 directions.
- Harris: cornerHarris returns a response map.
- Shi-Tomasi: goodFeaturesToTrack returns top-N corners directly.
- Use cornerSubPix for precise coordinates (calibration).

8. What's Next

ORB — keypoints + descriptors, the practical replacement for SIFT.

Lesson 3: ORB Keypoints & Descriptors

Module: 13 — Feature Detection & Matching

Duration: 35 minutes

Difficulty: Intermediate

Learning Objectives

- Use cv2.ORB_create to detect keypoints and compute descriptors.
- Distinguish keypoint vs descriptor.
- Visualise keypoints with cv2.drawKeypoints.

Prerequisites

- Module 13 Lesson 2

1. Why This Matters

ORB (Oriented FAST + Rotated BRIEF) is free, fast, and built into OpenCV. It produces rotation- and scale-invariant features that can be matched across images — the foundation for image alignment, panorama stitching, and basic visual recognition.

2. Concept

2.1 Keypoint vs descriptor

- Keypoint = a 2D location (often with a scale + orientation).
- Descriptor = a small vector that "describes" the local image patch around the keypoint, so two patches with similar content have similar descriptors.

You need both for matching: location says where, descriptor says what.

2.2 ORB pipeline (very brief)

- Detect FAST corners at multiple scales.
- Assign orientation per corner from intensity moments.
- Compute a 256-bit BRIEF descriptor at each keypoint, rotated to the keypoint's orientation.

2.3 OpenCV API

```
orb = cv2.ORB_create(nfeatures=500)
keypoints, descriptors = orb.detectAndCompute(gray, mask=None)
```

- keypoints: list of cv2.KeyPoint objects (with .pt, .angle, .size).
- descriptors: (N, 32) uint8 — each row is a 256-bit binary string.

2.4 ORB parameters

Param	Default	Meaning
nfeatures	500	max keypoints to return
scaleFactor	1.2	pyramid scale

nlevels	8	pyramid levels
edgeThreshold	31	edge pixels to ignore
WTA_K	2	descriptor type

2.5 Visualising

```
vis = cv2.drawKeypoints(img, keypoints, None,
                        flags=cv2.DRAW_MATCHES_FLAGS_DRAW_RICH_KEYPOINTS)
```

RICH_KEYPOINTS draws circles with size and orientation.

2.6 SIFT vs ORB (briefly)

- SIFT — patented for years (free since 2020); higher quality matches; slower; descriptor is 128 floats.
- ORB — always free; fast; binary descriptor; slightly less accurate.

For most practical work, ORB is fine. SIFT is preferred when you need more reliable matches and can afford the time.

3. Code Examples

Example 1: Detect ORB keypoints

```
import cv2
from skimage.data import astronaut

img = cv2.cvtColor(astronaut(), cv2.COLOR_RGB2BGR)
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

orb = cv2.ORB_create(nfeatures=500)
kp, desc = orb.detectAndCompute(gray, None)
print("keypoints:", len(kp))
print("descriptor shape:", desc.shape, desc.dtype)

vis = cv2.drawKeypoints(img, kp, None,
                        flags=cv2.DRAW_MATCHES_FLAGS_DRAW_RICH_KEYPOINTS)
cv2.imwrite("astro_orb_kp.png", vis)
```

Expected Output:

```
keypoints: 500
descriptor shape: (500, 32) uint8
```

Image shows ~500 circles of various sizes covering distinctive areas (eyes, helmet text, suit edges).

Example 2: Inspect a keypoint

```
import cv2
from skimage.data import astronaut

gray = cv2.cvtColor(cv2.cvtColor(astronaut(), cv2.COLOR_RGB2BGR),
cv2.COLOR_BGR2GRAY)
```

```
kp, _ = cv2.ORB_create(50).detectAndCompute(gray, None)
k = kp[0]
print(f"pt = ({k.pt[0]:.1f}, {k.pt[1]:.1f})  size = {k.size:.1f}  angle =
{k.angle:.1f}  response = {k.response:.3f}")
```

Expected Output (varies):

```
pt = (45.4, 312.7)  size = 31.0  angle = 75.2  response = 0.000178
```

Example 3: ORB on two views — same image

```
import cv2, numpy as np
from skimage.data import astronaut

img = cv2.cvtColor(astronaut(), cv2.COLOR_RGB2BGR)
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

# Slightly rotated version
h, w = gray.shape
M = cv2.getRotationMatrix2D((w/2, h/2), 15, 1.0)
rotated = cv2.warpAffine(gray, M, (w, h))

orb = cv2.ORB_create(500)
kp1, d1 = orb.detectAndCompute(gray, None)
kp2, d2 = orb.detectAndCompute(rotated, None)
print(f"original: {len(kp1)} kp, rotated: {len(kp2)} kp")
```

Expected Output: Both around 500 keypoints; ORB recovers most after rotation.

Example 4: Tune nfeatures

```
import cv2
from skimage.data import coffee

gray = cv2.cvtColor(cv2.cvtColor(coffee(), cv2.COLOR_RGB2BGR), cv2.COLOR_BGR2GRAY)
for n in [50, 200, 1000]:
    kp, _ = cv2.ORB_create(n).detectAndCompute(gray, None)
    print(f"nfeatures={n}: {len(kp)} keypoints returned")
```

Expected Output: Larger nfeatures returns more keypoints (up to what ORB can find).

4. Parameter Sensitivity

Param	Effect
nfeatures	how many keypoints to return
scaleFactor	scale-space granularity
edgeThreshold	excludes near-edge keypoints

5. Common Mistakes

Mistake	What Happens	Fix
Pass color image	Works but gray is preferred	Convert to grayscale
Forget desc may be None if no keypoints	NoneType errors downstream	Check if desc is None or len(kp) == 0:

Use cv2.SIFT_create() thinking it's free in old OpenCV	May fail on older builds	OpenCV ≥ 4.4 has it free

6. Practice Tasks

- Detect 500 ORB keypoints on astronaut(); visualise with RICH_KEYPOINTS.
- Inspect the first keypoint's pt, size, angle, response.
- Rotate the image 15°; detect again; compare keypoint count.

7. Key Takeaways

- ORB detects keypoints AND computes a 256-bit descriptor per point.
- Always free, fast, rotation-invariant.
- orb.detectAndCompute(gray, None) is the standard call.
- Pair with a binary matcher (next lesson).

8. What's Next

Matching descriptors between two images.

Lesson 4: Feature Matching

Module: 13 — Feature Detection & Matching

Duration: 35 minutes

Difficulty: Intermediate

Learning Objectives

- Match ORB descriptors with cv2.BFMatcher.
- Filter matches with Lowe's ratio test.
- Visualise matches with cv2.drawMatches.
- Use cv2.findHomography to recover the geometric relationship.

Prerequisites

- Module 13 Lesson 3

1. Why This Matters

Matching is the bridge from keypoints to actual alignment / recognition / stitching. ORB + BF + ratio test + homography is the recipe for panorama stitching, image registration, and basic visual SLAM front-ends.

2. Concept

2.1 Brute Force matcher

For each descriptor in image A, find the nearest descriptor in image B by distance.

For ORB (binary descriptors), use Hamming distance:

```
bf = cv2.BFMatcher(cv2.NORM_HAMMING, crossCheck=False)
matches = bf.knnMatch(desc1, desc2, k=2)
```

k=2 requests the 2 nearest neighbours — needed for the ratio test.

2.2 Lowe's ratio test

A match is "good" if the best neighbour is much better than the second-best. Otherwise the match is ambiguous (texture, repeats).

```
good = []
for m, n in matches:
    if m.distance < 0.75 * n.distance:
        good.append(m)
```

Typical threshold 0.7-0.8.

2.3 crossCheck=True alternative

Simpler than ratio: each match must be mutual (A's best in B and B's best in A). Use with bf.match, not bf.knnMatch:

```
bf = cv2.BFMatcher(cv2.NORM_HAMMING, crossCheck=True)
matches = bf.match(desc1, desc2)
matches = sorted(matches, key=lambda m: m.distance)
```

2.4 Drawing matches

```
img_matches = cv2.drawMatches(img1, kp1, img2, kp2, good, None,
                              flags=cv2.DRAW_MATCHES_FLAGS_NOT_DRAW_SINGLE_POINTS)
```

2.5 Homography from matches

If both images show the same planar surface, the geometric map between them is a 3×3 homography (M5 L3):

```
src_pts = np.float32([kp1[m.queryIdx].pt for m in good]).reshape(-1, 1, 2)
dst_pts = np.float32([kp2[m.trainIdx].pt for m in good]).reshape(-1, 1, 2)
H, mask = cv2.findHomography(src_pts, dst_pts, cv2.RANSAC, 5.0)
```

RANSAC filters out outlier matches. mask marks which matches survived.

2.6 Use cases

- Panorama stitching (M16 project): find H between overlapping photos, warp one onto the other.
- Image registration: align satellite photos taken months apart.
- Object detection (limited): given a template object, find it in a scene by matching ORB features and checking H consistency.

3. Code Examples

Example 1: Match ORB between original and rotated

```
import cv2, numpy as np
from skimage.data import astronaut

img1 = cv2.cvtColor(astronaut(), cv2.COLOR_RGB2BGR)
gray1 = cv2.cvtColor(img1, cv2.COLOR_BGR2GRAY)

# Slightly rotated version
h, w = gray1.shape
M = cv2.getRotationMatrix2D((w/2, h/2), 20, 1.0)
gray2 = cv2.warpAffine(gray1, M, (w, h))

orb = cv2.ORB_create(500)
kp1, d1 = orb.detectAndCompute(gray1, None)
kp2, d2 = orb.detectAndCompute(gray2, None)

bf = cv2.BFMatcher(cv2.NORM_HAMMING, crossCheck=False)
matches = bf.knnMatch(d1, d2, k=2)
good = [m for m, n in matches if m.distance < 0.75 * n.distance]
print(f"keypoints: {len(kp1)} / {len(kp2)}   raw matches: {len(matches)}   good:
{len(good)}")

img2 = cv2.cvtColor(gray2, cv2.COLOR_GRAY2BGR)
```

```
vis = cv2.drawMatches(img1, kp1, img2, kp2, good[:30], None,
                      flags=cv2.DRAW_MATCHES_FLAGS_NOT_DRAW_SINGLE_POINTS)
cv2.imwrite("matches.png", vis)
```

Expected Output: Console shows ~100+ good matches. Image is two views side-by-side with green lines connecting matching keypoints.

Example 2: crossCheck path

```
import cv2
from skimage.data import astronaut

gray = cv2.cvtColor(cv2.cvtColor(astronaut(), cv2.COLOR_RGB2BGR),
cv2.COLOR_BGR2GRAY)
orb = cv2.ORB_create(500)
kp1, d1 = orb.detectAndCompute(gray, None)
kp2, d2 = orb.detectAndCompute(gray, None)

bf = cv2.BFMatcher(cv2.NORM_HAMMING, crossCheck=True)
matches = sorted(bf.match(d1, d2), key=lambda m: m.distance)
print(f"matches: {len(matches)}   best distance: {matches[0].distance}")
```

Expected Output: Same image matched to itself → all keypoints match perfectly with distance 0.

Example 3: Homography between matched views

```
import cv2, numpy as np
from skimage.data import astronaut

img1 = cv2.cvtColor(astronaut(), cv2.COLOR_RGB2BGR)
gray1 = cv2.cvtColor(img1, cv2.COLOR_BGR2GRAY)
h, w = gray1.shape
M = cv2.getRotationMatrix2D((w/2, h/2), 20, 1.0)
gray2 = cv2.warpAffine(gray1, M, (w, h))
img2  = cv2.cvtColor(gray2, cv2.COLOR_GRAY2BGR)

orb = cv2.ORB_create(1000)
kp1, d1 = orb.detectAndCompute(gray1, None)
kp2, d2 = orb.detectAndCompute(gray2, None)

bf = cv2.BFMatcher(cv2.NORM_HAMMING, crossCheck=False)
matches = bf.knnMatch(d1, d2, k=2)
good = [m for m, n in matches if m.distance < 0.75 * n.distance]
print(f"good: {len(good)}")

if len(good) >= 4:
    src = np.float32([kp1[m.queryIdx].pt for m in good]).reshape(-1, 1, 2)
    dst = np.float32([kp2[m.trainIdx].pt for m in good]).reshape(-1, 1, 2)
    H, mask = cv2.findHomography(src, dst, cv2.RANSAC, 5.0)
    print("inliers:", int(mask.sum()), "/", len(good))
    print("H =\n", H)
```

Expected Output: Most matches are inliers (RANSAC keeps the consistent ones). Homography matrix is roughly a rotation by 20° around the centre.

4. Parameter Sensitivity

Param	Effect
Ratio threshold	0.7 strict, 0.8 lenient
nfeatures (ORB)	more keypoints → more potential matches
RANSAC threshold	larger = more inliers but less precise H

5. Common Mistakes

Mistake	What Happens	Fix
cv2.NORM_L2 with ORB	Wrong distance	Use NORM_HAMMING for binary descriptors
Skip ratio test	Tons of bogus matches	Always apply Lowe's
< 4 good matches for findHomography	Not enough constraints	Check len(good) ≥ 4
Treat all matches as correct after RANSAC	Outliers may still slip in	Use mask to filter

6. Practice Tasks

- Rotate astronaut() 20°. Match ORB descriptors; count good.
- Match an image to itself; show all matches have distance 0.
- Compute a homography from matched ORBs; print inlier count.

7. Key Takeaways

- cv2.BFMatcher(cv2.NORM_HAMMING) for ORB.
- Use knnMatch(k=2) + Lowe's ratio test (0.75).
- cv2.findHomography(..., cv2.RANSAC) survives outliers.
- ORB + BF + RANSAC = the classic stitching / registration recipe.

8. What's Next

Hough transform — detect lines and circles without matching.

Lesson 5: Hough Transform — Lines & Circles

Module: 13 — Feature Detection & Matching

Duration: 35 minutes

Difficulty: Intermediate

Learning Objectives

- Detect straight lines with cv2.HoughLines and cv2.HoughLinesP.
- Detect circles with cv2.HoughCircles.
- Tune accumulator threshold and minimum line length.

Prerequisites

- Module 9 (Canny), Module 13 Lessons 1-4.

1. Why This Matters

When you need parametric shapes (straight lines, circles), Hough is the classic tool — robust to gaps, occlusions, and noise. Used everywhere from lane detection to coin counting.

2. Concept

2.1 The accumulator idea

Each edge pixel votes for all parametric shapes (lines / circles) it could lie on. After all votes, peaks in the parameter space correspond to actual shapes in the image.

For a line, the parameter space is (ρ, θ) — distance from origin and angle.

2.2 cv2.HoughLines — infinite lines

```
edges = cv2.Canny(gray, 50, 150)
lines = cv2.HoughLines(edges, rho=1, theta=np.pi/180, threshold=150)
# returns array of (rho, theta) pairs, shape (N, 1, 2)
```

Convert each (rho, theta) to two points for drawing:

```
for rho, theta in lines[:, 0]:
    a, b = np.cos(theta), np.sin(theta)
    x0, y0 = a * rho, b * rho
    p1 = (int(x0 + 1000 * (-b)), int(y0 + 1000 *  a))
    p2 = (int(x0 - 1000 * (-b)), int(y0 - 1000 *  a))
    cv2.line(img, p1, p2, (0, 255, 0), 2)
```

2.3 cv2.HoughLinesP — line segments (preferred)

Probabilistic Hough returns finite segments with endpoints — much more useful:

```
segs = cv2.HoughLinesP(edges, rho=1, theta=np.pi/180, threshold=80,
                       minLineLength=40, maxLineGap=10)
# returns (N, 1, 4) of (x1, y1, x2, y2)
```

Param	Meaning
threshold	min votes for a line
minLineLength	discard segments shorter than this
maxLineGap	bridge gaps up to this length

2.4 cv2.HoughCircles

```
circles = cv2.HoughCircles(gray, cv2.HOUGH_GRADIENT, dp=1, minDist=20,
                           param1=100, param2=30, minRadius=10, maxRadius=80)
# returns (1, N, 3) of (x, y, r)
```

Param	Meaning
dp	inverse ratio of accumulator resolution to image (1 = same, 2 = half)
minDist	min distance between detected circle centres
param1	Canny high threshold (low = param1 / 2)
param2	accumulator threshold; smaller = more circles
minRadius, maxRadius	radius range

Pre-blur is essential — circles are noise-sensitive.

2.5 When to use Hough

Use	Notes
Lane detection	HoughLinesP with line angle filter
Coin / cell detection (round)	HoughCircles
Detecting structured shapes (paper edges)	HoughLinesP to find dominant lines
Generalised parametric matching	Generalised Hough (advanced)

2.6 Failure modes

- Noisy edges → too many spurious lines/circles. Increase threshold; pre-blur.
- Circle params off → no detections. Tune param2, radius range.
- Too many duplicate lines → use NMS (cluster nearby (ρ, θ)).

3. Code Examples

Example 1: HoughLinesP on a chessboard

```
import cv2, numpy as np
from skimage.data import checkerboard

img = (checkerboard().astype(np.uint8) * 255) if checkerboard().max() <= 1 else
checkerboard()
edges = cv2.Canny(img, 50, 150)
segs = cv2.HoughLinesP(edges, 1, np.pi/180, threshold=80,
                       minLineLength=40, maxLineGap=10)
vis = cv2.cvtColor(img, cv2.COLOR_GRAY2BGR)
for x1, y1, x2, y2 in segs[:, 0]:
    cv2.line(vis, (x1, y1), (x2, y2), (0, 0, 255), 2)
print("segments:", len(segs))
cv2.imwrite("checker_hlines.png", vis)
```

Expected Output: Checkerboard with red lines drawn along its grid edges.

Example 2: HoughCircles on coins

```
import cv2, numpy as np
from skimage.data import coins

gray = cv2.medianBlur(coins(), 5)
circles = cv2.HoughCircles(gray, cv2.HOUGH_GRADIENT, dp=1, minDist=30,
                           param1=100, param2=30, minRadius=20, maxRadius=40)
print("circles:", 0 if circles is None else circles.shape[1])
vis = cv2.cvtColor(coins(), cv2.COLOR_GRAY2BGR)
if circles is not None:
    for x, y, r in circles[0].astype(int):
        cv2.circle(vis, (x, y), r, (0, 255, 0), 2)
        cv2.circle(vis, (x, y), 2, (0, 0, 255), 3)
cv2.imwrite("coins_hcircles.png", vis)
```

Expected Output: Coin image with green circles drawn around each detected coin (radius approximately matching coin radius).

Example 3: Tune HoughLinesP threshold

```
import cv2, numpy as np
from skimage.data import camera

edges = cv2.Canny(camera(), 60, 150)
for t in [50, 100, 200]:
    segs = cv2.HoughLinesP(edges, 1, np.pi/180, threshold=t, minLineLength=20,
maxLineGap=5)
    n = 0 if segs is None else len(segs)
    print(f"threshold={t}: {n} segments")
```

Expected Output: Lower threshold → more segments. Higher threshold → only the strongest lines survive.

Example 4: Detect lane-like lines (horizontal filter)

```
import cv2, numpy as np

# Synthesize a "road" image
img = np.full((300, 600, 3), 100, dtype=np.uint8)
cv2.line(img, (50, 250), (300, 100), (255, 255, 255), 4)
cv2.line(img, (550, 250), (300, 100), (255, 255, 255), 4)
cv2.line(img, (100, 280), (500, 280), (200, 200, 200), 2)

gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
edges = cv2.Canny(gray, 50, 150)
segs = cv2.HoughLinesP(edges, 1, np.pi/180, 50, minLineLength=50, maxLineGap=20)

for x1, y1, x2, y2 in segs[:, 0]:
    angle = np.degrees(np.arctan2(y2 - y1, x2 - x1))
    if abs(angle) > 20:                                  # keep slanted lines (lanes)
        cv2.line(img, (x1, y1), (x2, y2), (0, 0, 255), 2)
cv2.imwrite("lanes.png", img)
```

Expected Output: Only the slanted "lane" lines outlined in red; the near-horizontal stripe is ignored.

4. Parameter Sensitivity

Param	Direction
threshold (lines)	higher = fewer lines
minLineLength	larger = filter short noise
maxLineGap	larger = bridge bigger gaps
param2 (circles)	smaller = more circles
minRadius/maxRadius	narrow range = faster + less noise

5. Common Mistakes

Mistake	What Happens	Fix
Skip Canny before HoughLines	No edges to vote	Run Canny first
Wrong radius range for circles	No detections	Inspect typical radius first
Skip pre-blur for circles	Noisy false positives	medianBlur(5) is a safe default
HoughLines vs HoughLinesP confusion	Different output formats	Prefer LinesP for segments

6. Practice Tasks

- HoughLinesP on a checkerboard; draw red lines.
- HoughCircles on coins(); draw a green circle on each.
- Filter to keep only slanted "lane" lines in a synthetic road scene.

7. Key Takeaways

- Hough = voting in parameter space.
- HoughLinesP for line segments; HoughCircles for circles.
- Always pre-process: Canny for lines, blur for circles.
- Tune thresholds for sensitivity.

8. What's Next

End of Module 13. Next: Module 14 — Frequency Domain (DFT).

Module 13 — Exercises

15 exercises.

13.1 Template Matching

- (E) Crop 50×50 from astronaut(); match back; verify max_loc.
- (M) Multi-match on synthetic scene with 3 copies of an "X".
- (H) Compare TM_SQDIFF / TM_CCORR_NORMED / TM_CCOEFF_NORMED on the same input.

13.2 Harris / Shi-Tomasi

- (E) Harris on checkerboard(); show red corner dots.
- (M) goodFeaturesToTrack (100) on astronaut().
- (H) Compare qualityLevel ∈ {0.005, 0.05, 0.2}.

13.3 ORB

- (E) ORB(500) on astronaut(); draw RICH_KEYPOINTS.
- (M) Inspect kp[0].pt, .size, .angle, .response.
- (H) Rotate image 15°; detect again; compare kp counts.

13.4 Matching

- (E) Rotate astronaut() 20°. Match ORB; count good.
- (M) Match an image to itself; verify distance 0.
- (H) Estimate homography with RANSAC; print inlier count.

13.5 Hough

- (E) HoughLinesP on checkerboard(); draw red lines.
- (M) HoughCircles on coins(); draw green circles.
- (H) Filter lanes by angle from a synthetic road scene.

Module 13 — Quiz

10 MCQs.

Q1

For brightness-invariant template matching use: A. TM_SQDIFF B. TM_CCORR C. TM_CCOEFF_NORMED D. TM_CCORR_NORMED

Answer: C (L1)

Q2

Template matching is invariant to: A. brightness (with NORMED) but not rotation/scale B. all transforms C. rotation D. nothing

Answer: A (L1)

Q3

Harris corner = strong gradient in: A. one direction B. two directions C. four directions D. all directions

Answer: B (L2)

Q4

cv2.goodFeaturesToTrack returns: A. all corners B. response map C. top-N strongest corners D. only the best

Answer: C (L2)

Q5

ORB descriptor is: A. 128 floats B. 256-bit binary (32 bytes uint8) C. JPEG-encoded D. RGB color

Answer: B (L3)

Q6

For ORB matching, the right distance metric is: A. NORM_L2 B. NORM_HAMMING C. NORM_INF D. NORM_L1

Answer: B (L4)

Q7

Lowe's ratio test discards matches where: A. best > 0.75 × second-best B. distance > 0.75 C. crosscheck fails D. RANSAC rejects

Answer: A (L4)

Q8

cv2.findHomography(..., cv2.RANSAC, 5.0): A. requires exact data B. tolerates outliers C. fits a line D. returns a 2x2

Answer: B (L4)

Q9

For finite line segments, use: A. HoughLines B. HoughLinesP C. HoughCircles D. matchTemplate

Answer: B (L5)

Q10

For circle detection, the typical pre-processing is: A. Canny B. medianBlur C. erosion D. equalizeHist

Answer: B (L5)

Module 13 — Assignment: Logo Finder

Estimated time: 2 hours

Combines: Module 13 + Modules 7, 9

Brief

Given a small "logo" template and a larger "scene", find the logo in the scene using:

- Template matching (rigid).
- ORB + matching + homography (rotation/scale invariant).

Compare results.

Requirements

- find_logo.py SCENE LOGO [--mode template|orb] [--output OUT]
- template mode: cv2.matchTemplate + threshold; draw bounding box of best match.
- orb mode: ORB on both → BFMatcher + Lowe ratio → findHomography → warp logo bbox into scene; draw resulting quadrilateral.
- Print number of matches (orb) or best score (template).
- Save annotated scene.

Constraints

- Modules 1-13 only.
- Use argparse.
- Handle the case "no good match" gracefully.

Expected Output

```
$ python find_logo.py scene.png logo.png --mode template
best score: 0.92  at (x=325, y=180)
saved -> scene_template.png

$ python find_logo.py scene.png logo.png --mode orb
good matches: 84   inliers: 71
saved -> scene_orb.png
```

Grading Rubric

Criterion	Weight
Template path	30%
ORB + matching path	35%
findHomography use	15%
CLI + reporting	10%
Style	10%

Submission

Save as assignment_module13.py.

Module 13 — Mini Project: Coin Counter via Hough

Overview

Count circular coins in a photo using cv2.HoughCircles. Output an annotated image with each coin numbered and the total.

Concepts Used

- cv2.medianBlur (M7 L4)
- cv2.HoughCircles (M13 L5)
- Drawing (M3 L2)

Features

- count_coins.py INPUT [--minR 20] [--maxR 60] [--p2 30] [--output OUT]
- Pre-blur with medianBlur(5).
- Detect circles via HOUGH_GRADIENT.
- Number each detected coin; print total.

Starter Code

count_coins.py:

```
import sys, cv2, numpy as np, argparse

def main():
    ap = argparse.ArgumentParser()
    ap.add_argument("input")
    ap.add_argument("--minR", type=int, default=20)
    ap.add_argument("--maxR", type=int, default=60)
    ap.add_argument("--p2",   type=int, default=30)
    ap.add_argument("--output", default="coins_counted.png")
    args = ap.parse_args()

    img = cv2.imread(args.input)
    if img is None:
        raise SystemExit("can't load: " + args.input)
    gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
    gray = cv2.medianBlur(gray, 5)

    circles = cv2.HoughCircles(gray, cv2.HOUGH_GRADIENT, dp=1, minDist=args.minR,
                               param1=100, param2=args.p2,
                               minRadius=args.minR, maxRadius=args.maxR)
    if circles is None:
        print("no circles found"); return

    for i, (x, y, r) in enumerate(circles[0].astype(int), 1):
        cv2.circle(img, (x, y), r, (0, 255, 0), 2)
        cv2.putText(img, str(i), (x - 10, y + 5),
                    cv2.FONT_HERSHEY_SIMPLEX, 0.6, (0, 0, 255), 2)
    print("coins:", circles.shape[1])
```

```
    cv2.imwrite(args.output, img)

if __name__ == "__main__":
    main()
```

Expected Output

```
$ python count_coins.py datasets/starter/coins.png
coins: 24
```

coins_counted.png shows green circles around each coin, each numbered 1..N.

Extension Challenges

- Add trackbars for live param tuning before commit.
- Cluster coins by radius into 2 "denominations".
- Robust mode: try several param2 values and pick one that gives circles within expected count range.

Grading Rubric

Criterion	Weight
Circle detection works	35%
Pre-blur applied	10%
Annotation + numbering	25%
CLI / argparse	15%
Style	15%